

LLM Local Chat

IA Local para PCs Fracos

Interface de chat com IA local otimizada para PCs fracos e antigos.

Roda modelos 12B em CPU com apenas 2-4GB de RAM.

Autor: Gabriel Garcia

Data: Setembro 2026

Licenca: MIT

Sumario

1. Visao Geral
2. Arquitetura
3. Agentes de IA
4. Niveis de Potencia
5. Interface Web
- 6.Codigo-Fonte
7. Testes
8. Performance
9. Instalacao
10. Contribuicao

1. Visao Geral

O LLM Local Chat e uma interface de chat com IA local, otimizada para PCs fracos e antigos.

Diferente de solucoes como ChatGPT, Claude ou Gemini, que rodam em servidores na nuvem, o LLM Local Chat

roda 100% no seu computador, sem necessidade de internet, sem custos de API e com privacidade total.

Caracteristicas principais:

- 100% Local: sem internet, sem API, sem custos
- Otimizado para CPU: nao precisa de GPU
- Modelos 12B comprimidos: quantizacao Q4_K_M/Q2_K
- Baixo consumo de RAM: 2-4GB suficiente
- Interface web estilo ChatGPT: familiar e intuitiva
- Privacidade total: dados nunca saem do seu PC
- Streaming: respostas em tempo real
- Roteamento automatico: detecta o agente ideal por mensagem
- Niveis de potencia: ajuste velocidade vs qualidade

O projeto utiliza o Ollama como backend de inferencia, que gerencia modelos quantizados GGUF otimizados para execucao em CPU com uso intensivo de disco (swap) quando a RAM e insuficiente.

2. Arquitetura

O projeto segue uma arquitetura modular com 4 camadas principais:

1. Camada de Dados (models/)
 - Modelos GGUF quantizados (Q4_K_M, Q2_K)
 - Cache de sessoes em memoria
 - Arquivos de configuracao (.env)
2. Camada de Cliente (core/client.py)
 - OllamaClient: comunicacao com API do Ollama
 - ChatManager: gerenciamento de sessoes e historico
 - AgentRouter: roteamento automatico de agentes
 - ChatMessage/ChatResponse: contratos de dados
3. Camada Web (web/)
 - server.py: FastAPI backend com endpoints REST
 - templates/index.html: interface web estilo ChatGPT
 - static/style.css: estilos CSS responsivos
4. Camada de Interface (cli.py)
 - CLI com Typer para comandos de terminal
 - Comandos: start, list-models, pull, chat

Fluxo de dados:

Usuario -> Interface Web -> FastAPI -> ChatManager -> AgentRouter -> OllamaClient -> Ollama -> Modelo GGUF

3. Agentes de IA

O LLM Local Chat possui 4 agentes de IA, cada um com especialidade diferente:

1. ATHENA (Assistente Geral)

- Base: Qwen 2.5 12B
- RAM minima: 4GB
- Descricao: Assistente geral - conversa, analise, codigo
- System prompt: "You are Athena, a helpful general assistant..."

2. MERCATUS (Vendas e Trading)

- Base: Qwen 2.5 12B
- RAM minima: 4GB
- Descricao: Especialista em vendas e trading
- System prompt: "You are Mercatus, a sales and trading expert..."

3. CORTEX (Orquestracao)

- Base: Qwen 2.5 12B
- RAM minima: 4GB
- Descricao: Orquestracao multi-maquina
- System prompt: "You are Cortex, a multi-machine orchestration expert..."

4. LOCALMIND (Ultra Leve)

- Base: Qwen 2.5 4B
- RAM minima: 2GB
- Descricao: Ultra leve - roda em qualquer PC
- System prompt: "You are LocalMind, a lightweight assistant..."

Roteamento Automatico:

O AgentRouter detecta automaticamente o agente ideal baseado em palavras-chave:

- Mercatus: sales, trading, revenue, profit, market, investment, stock, trade...
- Cortex: cluster, orchestration, distributed, server, node, task, schedule...
- Athena: padrao (quando nenhum outro agente e detectado)

4. Niveis de Potencia

O sistema possui 3 niveis de potencia, inspirados no Hermes Agent:

1. MINIMAL

- Modelo: LocalMind 4B
- Max tokens: 1024
- Temperatura: 0.5
- Descricao: Mais rapido, menor qualidade - para PCs muito fracos
- Uso: PCs com 2GB RAM, Celeron, tarefas simples

2. MEDIUM (Recomendado)

- Modelo: Athena 12B
- Max tokens: 4096
- Temperatura: 0.7
- Descricao: Equilibrio entre velocidade e qualidade
- Uso: PCs com 4-8GB RAM, i3/i5, uso geral

3. ULTRA

- Modelo: Athena 12B
- Max tokens: 8192
- Temperatura: 0.9
- Descricao: Mais qualidade, mais lento - para tarefas complexas
- Uso: PCs com 8GB+ RAM, i5/i7, tarefas complexas

O nivel de potencia pode ser alterado em tempo real pela interface web, sem necessidade de reiniciar o servidor.

5. Interface Web

A interface web e inspirada no ChatGPT, com design moderno e responsivo:

Componentes:

- Sidebar: selecao de agente e nivel de potencia
- Area de chat: mensagens com streaming em tempo real
- Input: textarea com auto-resize e atalhos de teclado
- Status: indicador de conexao com Ollama

Funcionalidades:

- Nova conversa (botao no sidebar)
- Selecao de agente (Auto, Athena, Mercatus, Cortex, LocalMind)
- Selecao de potencia (Minimal, Medium, Ultra)
- Streaming de respostas em tempo real
- Formatacao markdown (negrito, italico, codigo, blocos de codigo)
- Sugestoes de mensagens na tela inicial
- Design responsivo (funciona em mobile)

Tecnologias:

- Backend: FastAPI + Uvicorn
- Frontend: HTML5 + CSS3 + JavaScript vanilla
- Streaming: Server-Sent Events (SSE)
- Templates: Jinja2

6.Codigo-Fonte

Estrutura do projeto:

```
llm-local-chat/
src/llm_local_chat/
  __init__.py      # Definicoes de agentes e niveis de potencia
core/
  __init__.py
  client.py        # OllamaClient, ChatManager, AgentRouter
web/
  __init__.py
  server.py        # FastAPI backend
  templates/
    index.html     # Interface web
  static/
    style.css      # Estilos CSS
utils/
  __init__.py
  logger.py        # Logging
  cli.py           # Interface CLI
tests/
  test_client.py   # Testes unitarios
docs/
models/            # Modelos GGUF (baixados via Ollama)
scripts/
.env.example
.gitignore
LICENSE
pyproject.toml
README.md
```

Arquivos principais:

- __init__.py: 3.5KB - Definicoes de agentes e niveis de potencia
- client.py: 10.6KB - Cliente Ollama + gerenciador de chat + roteamento
- server.py: 4.9KB - Servidor FastAPI
- index.html: 13.2KB - Interface web completa
- style.css: 7.0KB - Estilos CSS
- cli.py: 4.1KB - Interface CLI
- test_client.py: 2.0KB - Testes unitarios

7. Testes

O projeto possui 7 testes unitarios, todos passando:

1. TestChatMessage::test_create_message - PASSED
Testa criacao de mensagem com role e conteudo
2. TestChatMessage::test_to_dict - PASSED
Testa serializacao de mensagem para dicionario
3. TestChatManager::test_create_session - PASSED
Testa criacao de sessao de chat
4. TestChatManager::test_add_message - PASSED
Testa adicao de mensagem a uma sessao
5. TestChatManager::test_clear_session - PASSED
Testa limpeza de sessao
6. TestOllamaClient::test_init - PASSED
Testa inicializacao do cliente Ollama
7. TestOllamaClient::test_init_custom - PASSED
Testa inicializacao com parametros customizados

Cobertura: 100% das classes principais (ChatMessage, ChatManager, OllamaClient)

Framework: pytest 8.4.2

Tempo de execucao: 0.17s

8. Performance

Performance esperada por hardware:

PC	RAM	CPU	Tokens/seg	Nivel recomendado
Celeron + 4GB	4GB	Celeron N4000	~2-5 t/s	Minimal
i3 + 8GB	8GB	i3-8100	~8-15 t/s	Medium
i5 + 8GB	8GB	i5-7360U	~10-20 t/s	Medium/Ultra
i7 + 16GB	16GB	i7-9700K	~20-35 t/s	Ultra

Otimizacoes para PCs fracos:

- Quantizacao Q4_K_M: reduz modelo de 24GB para ~7GB
- Quantizacao Q2_K: reduz modelo de 24GB para ~5GB
- Uso de mmap: mapeia modelo para disco, reduz RAM
- Swap: usa disco como RAM extra quando necessario
- N_THREADS: limita threads de CPU para evitar sobrecarga
- N_BATCH: tamanho de batch otimizado para CPU

Requisitos minimos:

- RAM: 2GB (LocalMind) ou 4GB (Athena)
- Disco: 3GB (LocalMind) ou 7GB (Athena)
- CPU: Qualquer x86_64 (recomendado: 2+ cores)
- SO: Linux, macOS ou Windows

9. Instalacao

Passo a passo para instalacao:

1. Instalar Ollama

macOS/Linux: `curl -fsSL https://ollama.com/install.sh | sh`

Windows: baixar de <https://ollama.com/download>

2. Baixar modelos

`ollama pull qwen2.5:12b` # Athena 12B (recomendado)

`ollama pull qwen2.5:4b` # LocalMind 4B (PCs fracos)

3. Instalar LLM Local Chat

`git clone https://github.com/UnloosedApple50/llm-local-chat.git`

`cd llm-local-chat`

`pip install -e ".[dev]"`

4. Iniciar

`llm-chat start`

5. Acessar

Abrir <http://localhost:8080> no navegador

Comandos CLI adicionais:

`llm-chat list-models` # Lista modelos disponiveis

`llm-chat pull <model>` # Baixa um modelo

`llm-chat chat <msg>` # Envia mensagem (modo CLI)

10. Contribuicao

Contribuicoes sao bem-vindas! Para contribuir:

1. Fork o repositorio
2. Crie uma branch para sua feature (git checkout -b feature/nova-feature)
3. Commit suas mudancas (git commit -m 'Adiciona nova feature')
4. Push para a branch (git push origin feature/nova-feature)
5. Abra um Pull Request

Padroes de codigo:

- Python 3.9+
- Type hints em todas as funcoes
- Docstrings em portugues
- Testes unitarios para novas funcionalidades
- Lint com ruff
- Type check com mypy

Licenca:

Este projeto e licenciado sob MIT License - veja o arquivo LICENSE para detalhes.

Repositorio:

<https://github.com/UnloosedApple50/llm-local-chat>

Contato:

Gabriel Garcia - UnloosedApple50